MASAREN ()

Impact Infrastructure as a Service

Technical Architecture Documentation

Version 2.0.0

The MASAREN Engineering Team

August 20, 2026

Core Principle: If an action cannot be verified, it cannot become an economic asset.

Abstract

MASAREN () represents the world's first **Impact Infrastructure as a Service (IIaaS)** platform, converting verified real-world field actions into immutable, measurable, and economically valuable digital assets. Built on modern microservices architecture with 35+ services, the platform combines Domain-Driven Design with event-driven principles to create a production-grade system supporting millions of users across Web and Mobile interfaces.

This document provides the comprehensive technical architecture, implementation patterns, and operational guidelines for the MASAREN platform. It serves as the definitive reference for technical implementation, deployment strategies, and system integration.

Key Differentiators:

- Event-Driven Architecture leveraging Kafka as source of truth
- Cross-platform deployment with React+TypeScript and Flutter applications
- Cloud-native AI with zero local inference
- Enterprise-grade security with RBAC and GDPR compliance
- Combined backend for simplified local development

Contents

1 Introduction

1.1 Mission and Purpose

MASAREN stands as the pioneering **Impact Infrastructure Platform** that transforms verified real-world field actions into digitally verifiable, quantifiable economic assets. The platform addresses a fundamental challenge in impact investing and value measurement by creating immutable digital representations of verified activities.

The platform operates at the intersection of:

- **Intention and Execution** – bridging planned objectives with field-level actions
- **Field Work and Digital Assets** – converting physical actions into digital value
- **Trust and Measurable Reputation** – building verifiable trust through data

1.2 Platform Vision

Verification-Centric Architecture Every asset creation requires multi-layered verification through AI, GPS, and human-in-the-loop review

Cross-Platform Consistency Web and mobile experiences maintain identical feature sets through shared backend APIs

Event-Driven Data Consistency Kafka serves as the single source of truth for all domain events with at-least-once delivery guarantees

Cloud-First AI Strategy No local LLM execution; all AI inference routes to cloud providers (OpenAI / Google Gemini)

Enterprise-Grade Security Zero-trust architecture with JWT-based authentication, RBAC, and comprehensive audit trails

Developer-First Design Comprehensive shared kernel, combined backend for local development, and full CI/CD pipelines

1.3 Core Architecture Principles

The MASAREN platform is built on the following engineering principles:

1. **Domain-Driven Design:** Complex business logic is organized into bounded contexts with ubiquitous language, aggregate patterns, and domain events.
2. **Clean/Hexagonal Architecture:** Separation of concerns places core business logic at the center, surrounded by infrastructure adapters.
3. **CQRS with Event Sourcing:** Commands and queries are separated with immutable event logs providing audit trails and data replication.

4. **Transactional Outbox Pattern:** Database transactions and Kafka event publishing are atomic to prevent data loss.

5. **Event-Driven Microservices:** 35+ independent services communicate through the Kafka event bus.

6. **Repository Pattern:** Abstract data access with domain-specific repositories ensures persistence ignorance.

7. **Specification Pattern:** Query building and filtering for complex data retrieval across bounded contexts.

2 System Architecture Overview

2.1 High-Level Architecture Layers

The MASAREN architecture follows a layered design with seven distinct layers:

1. **Presentation Layer:** React/TypeScript web applications and Flutter mobile app serving as the user-facing interfaces
2. **Experience Layer:** API Gateway (BFF pattern) at port 8080 providing routing, JWT authentication, rate limiting, and aggregation
3. **Business Layer:** 35+ domain services organized into bounded contexts (Identity, Execution, Verification, Reputation, Marketplace, etc.)
4. **AI Layer:** Cloud-based AI agents for vision verification, fraud detection, economist analysis, and graph synthesis
5. **Knowledge Layer:** Neo4j knowledge graph storing complex relationships between entities, programs, and impact assets
6. **Data Layer:** PostgreSQL+PostGIS for transactional data, Redis for caching/sessions, MinIO/S3 for evidence blobs
7. **Infrastructure Layer:** Docker/Kubernetes orchestration, Kafka event bus, monitoring with Prometheus/Grafana

2.2 Architecture Diagram

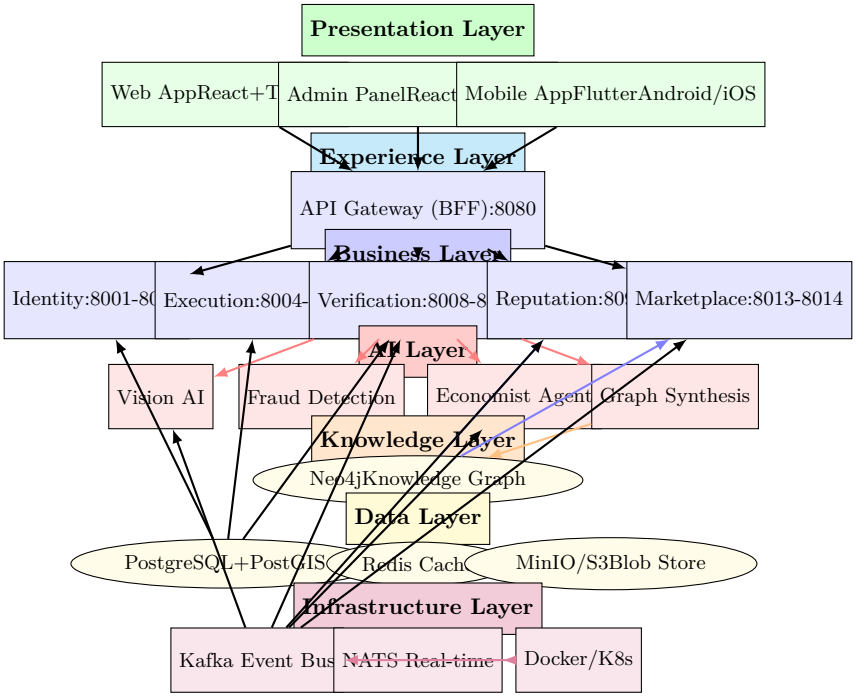


Figure 1: MASAREN Layered Architecture Diagram

2.3 Sync and Async Communication

The platform employs a dual-transport strategy:

Synchronous (REST) All service-to-service and client-to-service communication uses REST over HTTP via FastAPI. The API Gateway at port 8080 serves as the Backend-for-Frontend (BFF), handling routing, authentication, rate limiting, and response aggregation.

Asynchronous (Kafka + NATS) Domain events are published to Kafka topics with at-least-once delivery guarantees. The Transactional Outbox pattern ensures atomicity between database writes and event publication. NATS provides real-time fan-out for social features including chat, posts, and feed updates.

2.4 Database Strategy

The platform uses a polyglot persistence approach:

- **PostgreSQL + PostGIS:** Primary transactional store for all domain entities. PostGIS extension enables spatial queries for geolocation-based verification.
- **Neo4j + APOC:** Knowledge graph storing complex relationships between organizations, programs, tasks, users, and impact assets. Graph queries enable sophisticated reputation scoring and opportunity matching.
- **Redis 7:** Session storage, API rate limiting, and cache layer for frequently accessed data.
- **MinIO / S3:** Blob storage for evidence media files (images, videos, documents) uploaded during verification.
- **OpenSearch (planned):** Full-text search across programs, opportunities, and impact assets.

3 Technology Stack

3.1 Backend Technologies

Technology	Layer	Details
Python 3.11+	Runtime	Type-hinted, async-capable runtime for all microservices
FastAPI	Web Framework	OpenAPI 3.1 auto-documentation, dependency injection, async support
PostgreSQL 16 + PostGIS	Database	ACID-compliant relational data with spatial query extensions
Neo4j 5 + APOC + GDS	Graph Database	Knowledge graph, graph algorithms for scoring and matching
Redis 7	Cache	Session management, rate limiting, pub/sub messaging
Apache Kafka 3.7	Message Broker	KRaft mode (no Zookeeper) with 20+ event topics
NATS	Real-time	Fan-out messaging for social features and live updates
MinIO / S3	Object Storage	Blob storage for evidence media and attachments
OpenAI / Gemini	AI Inference	Cloud-based LLMs for vision, fraud detection, and analysis

3.2 Frontend Technologies

3.2.1 Web Application (apps/web)

Technology	Details
React 18.2 + TypeScript 5.3	Component-based UI framework with static typing
Vite 5.1	Fast build tool with HMR and optimized production bundling
React Router v6	Client-side routing with protected routes and role-based access
TanStack Query	Server state management with caching, refetching, and mutations
React Context	Client state management for auth, theme, and UI state
Tailwind CSS 3.4	Utility-first CSS framework with responsive design

Technology	Details
Axios	HTTP client with interceptors for automatic JWT refresh
i18n	Internationalization for English and Arabic (RTL) support
Google OAuth	Popup-based authentication flow with secure token handling
Vitest + RTL + Playwright	Unit, component, and E2E testing frameworks

3.2.2 Admin Panel (apps/admin)

Technology	Details
React + TypeScript	Same stack as web app with Mantine component library
Mantine UI	Rich component library for admin interfaces (tables, forms, modals)
Vite	Optimized build with separate configuration
nginx	Production serving with reverse proxy configuration

3.2.3 Mobile Application (apps/mobile)

Technology	Details
Flutter 3.22+	Cross-platform mobile framework with Dart 3
MVVM Pattern	Architecture pattern with Provider for state management
Provider Package	State management with ChangeNotifier and MultiProvider
Flutter Navigator	Custom bottom navigation with drawer for deep navigation
http Package	Pure Dart HTTP client with automatic JWT handling
shared_preferences	Local persistence for user preferences and cached data
flutter_secure_storage	Secure token storage using platform keychains
Material Design 3	Custom theme with responsive layouts and bilingual support
Flutter Test + Maestro	Unit, widget, and E2E integration testing

3.3 DevOps and Infrastructure

Tool	**Purpose**
Docker	Containerization for all services and infrastructure components
Docker Compose	Local development orchestration with 30+ services
Kubernetes	Production deployment with Helm charts and Kustomize
Prometheus + Grafana	Metrics collection, monitoring dashboards, and alerting
OpenTelemetry	Distributed tracing across microservices
GitHub Actions	CI/CD pipelines for build, test, and deployment automation
Terraform	Infrastructure as Code for cloud resource provisioning
nginx	Reverse proxy for frontend serving and API routing

4 Service Domain Architecture

The MASAREN platform comprises 35+ microservices organized into bounded contexts. Each domain encapsulates a distinct area of business capability with its own database schema, event contracts, and API surface.

4.1 Gateway Service

The API Gateway at port 8080 serves as the single entry point for all client requests. It implements the Backend-for-Frontend (BFF) pattern with the following responsibilities:

- **Routing:** Proxies requests to appropriate backend services based on URL paths
- **JWT Authentication:** Validates and decodes JWT tokens before forwarding requests
- **Rate Limiting:** Redis-backed rate limiting per client IP and user
- **Response Aggregation:** Combines responses from multiple services when needed
- **CORS Management:** Handles cross-origin requests from web and mobile clients
- **Error Mapping:** Standardizes error responses across all services

4.2 Identity Domain

The Identity domain manages all user authentication, authorization, and profile management. It is the foundation of the platform's security model.

4.2.1 Identity Service (port 8001)

The core user management service handling:

- User registration with email verification
- Profile creation and management (personal, personnel, institution)
- Email and phone verification workflows
- Password management and recovery
- User preference management

Key database tables: `users`, `user_profiles`, `verification_attempts`

4.2.2 Authentication Service (port 8002)

Handles all authentication flows:

- JWT token issuance and validation
- OAuth2 integration (Google OAuth with popup flow)

- Session management with refresh token rotation
- Multi-factor authentication support
- Token blacklisting for logout

Key database tables: `oauth_sessions`, `tokens`, `refresh_tokens`

4.2.3 Authorization Service (port 8003)

Role-based access control (RBAC) management:

- Role definitions (VOLUNTEER, ORG_ADMIN, ADMIN, SUPER_ADMIN)
- Permission assignment per role
- Resource-level authorization checks
- Organization-specific role scoping
- Permission inheritance and hierarchy

Key database tables: `roles`, `permissions`, `role_permissions`

4.3 Execution Domain

The Execution domain manages the core operational workflow of the platform: organizations create programs, programs contain tasks, and tasks are assigned to volunteers for execution.

4.3.1 Organization Service (port 8004)

Manages organizational entities and hierarchies:

- Organization creation and verification
- Organizational hierarchy (parent/child organizations)
- Membership management with role assignment
- Organization profiles with SDG alignment
- Institution-specific configurations

Key database tables: `organizations`, `affiliations`, `org_members`

4.3.2 Program Service (port 8005)

Program lifecycle management:

- Program creation with SDG (Sustainable Development Goal) mapping
- Program lifecycle states: DRAFT → ACTIVE → COMPLETED → ARCHIVED
- Enrollment management for volunteers

- Program metrics and progress tracking
- Budget and resource allocation

Key database tables: `programs`, `program_enrollments`, `sdg_mappings`

4.3.3 Task Service (port 8006)

Finite state machine for task management:

- Task creation within programs
- State machine: CREATED → ASSIGNED → IN_PROGRESS → SUBMITTED → VERIFIED → COMPLETED
- Assignment to volunteers with deadline management
- Location-based task discovery and matching
- Task dependencies and prerequisite chains

Key database tables: `tasks`, `task_assignments`, `task_states`

4.3.4 Workflow Service (port 8007)

Process orchestration and automation:

- Workflow definition with sequential and parallel steps
- Conditional branching based on verification outcomes
- Automated task creation based on workflow templates
- SLA monitoring and escalation
- Integration with verification and AI pipelines

Key database tables: `workflows`, `workflow_tasks`, `workflow_triggers`

4.4 Verification Domain

The Verification domain is the core differentiator of the MASAREN platform. It ensures that every claimed field action undergoes rigorous multi-layered verification before becoming an economic asset. This domain comprises six services working in concert.

4.4.1 Verification Service (port 8008)

Central verification engine that coordinates the verification pipeline:

- Orchestrates multi-layered verification (AI + GPS + human review)
- Implements configurable verification rules per program type

- Manages verification state machine: PENDING → IN_REVIEW → APPROVED/REJECTED/FLAGGED
- Calculates verification confidence scores
- Triggers fraud detection when anomalies are detected
- Routes to human-in-the-loop (HITL) review for edge cases

4.4.2 Evidence Service (port 8009)

Evidence capture and management:

- Multi-format evidence ingestion (images, videos, documents, audio)
- Versioning and chain of custody for all evidence items
- Metadata extraction (EXIF, timestamps, device info)
- Evidence integrity verification (hash checksums)
- Integration with MinIO/S3 for blob storage

4.4.3 GPS Service

Geolocation verification:

- GPS coordinate capture with timestamp binding
- Geofencing validation against program locations
- Location trajectory analysis for pattern verification
- Spatial queries using PostGIS
- Anti-spoofing detection for fake GPS data

4.4.4 Media Service

Media processing pipeline:

- Image and video compression and format conversion
- OCR (Optical Character Recognition) for document evidence
- Face detection and anonymization for privacy compliance
- Thumbnail generation for gallery views
- Content moderation flagging

4.4.5 Audit Service

Comprehensive audit trail management:

- Immutable audit logs for all verification events
- Compliance reporting for regulatory requirements
- Tamper-evident log storage
- Queryable audit trail for investigations
- GDPR compliance data management

4.4.6 Submission Service

Submission intake and validation:

- Evidence submission intake from mobile and web clients
- Schema validation against program requirements
- Preliminary automated validation checks
- Queue management for verification processing
- Submission status tracking and notifications

4.5 AI Domain

The AI domain provides cloud-based intelligent agents that augment the verification pipeline. All AI inference is performed exclusively through cloud APIs (OpenAI GPT-4o/GPT-4o Vision and Google Gemini Pro/Gemini Flash) – no local LLM execution occurs on the deployment hardware.

4.5.1 Vision AI Service

Input Event: `evidence_uploaded`

Cloud Model: GPT-4o Vision / Gemini Pro

Output Event: `evidence_verified`

Responsibilities: Visual evidence verification, object detection in images/videos, OCR on document evidence, quality assessment of submitted media, content moderation flagging

4.5.2 Fraud Detection Service

Input Events: `location_reached`, `evidence_uploaded`

Cloud Model: GPT-4o / Gemini Flash

Output Event: `fraud_detected`

Responsibilities: Pattern-based fraud detection, anomaly detection in submission patterns, GPS spoofing analysis, duplicate evidence detection, behavioral analysis of user submission patterns

4.5.3 Economist Agent

Input Event: `evidence_verified` (APPROVED status)

Cloud Model: GPT-4o / Gemini Pro

Output Event: `impact_asset_created`

Responsibilities: Impact value calculation based on verified evidence, SDG contribution quantification, economic value assignment, asset metadata generation, PMS (Personal Metric Score) calculation

4.5.4 Graph Synthesis Agent

Input Event: `impact_asset_created`

Cloud Model: GPT-4o / Gemini Pro

Output Event: `pms_updated` + Neo4j Cypher queries

Responsibilities: Knowledge graph updates in Neo4j, relationship inference between entities, impact propagation through graph, automated graph maintenance and deduplication

4.5.5 AI Orchestrator (port 8070)

The orchestrator coordinates the POAI (Proof of AI) deliberative crew:

- Manages the execution pipeline for all AI agents
- Implements crew coordination with agent handoffs
- Handles retry logic and fallback strategies
- Provides a unified API for AI operations
- Logs all AI decisions for audit and improvement

4.5.6 Additional AI Modules

The following AI agents are implemented but not yet wired into docker-compose.yml:

- **Auditor Agent:** Compliance verification and regulatory review
- **Broker Agent:** Opportunity matching and recommendation
- **Planner Agent:** Program planning and resource optimization
- **Recommendation Agent:** Personalized user recommendations
- **Speech AI Service:** Voice-based evidence submission processing
- **Compliance Service:** Automated regulatory compliance checking
- **Media Distribution Service:** Automated media publishing workflows

4.6 Reputation Domain

The Reputation domain quantifies and tracks the impact and reputation of individuals and organizations within the platform.

4.6.1 Impact Score Service (port 8094)

Individual impact scoring:

- Personal Metric Score (PMS) calculation based on verified actions
- SDG contribution tracking per user
- Historical score trends and projections
- Score breakdown by category and program

4.6.2 Organization Score Service

Organizational reputation management:

- Organizational Impact Index (OII) calculation
- Aggregate organizational performance metrics
- Peer comparison and benchmarking
- Transparency and trust scoring

4.6.3 Portfolio Service

Impact portfolio management:

- Individual impact asset portfolio tracking
- Portfolio diversification analysis
- Impact asset valuation over time
- Portfolio export and sharing capabilities

4.7 Marketplace Domain

The Marketplace domain connects impact creators with opportunity providers.

4.7.1 Opportunity Service (port 8013)

Opportunity lifecycle management:

- Opportunity creation by organizations
- Opportunity states: DRAFT → OPEN → IN_PROGRESS → FILLED → CLOSED
- Search and discovery with filters (location, SDG, skills)
- Application and matching workflows

4.7.2 Matching Service (port 8014)

Intelligent matching between opportunities and talent:

- Skill-based matching algorithms
- Location proximity matching
- Reputation score integration
- Availability and capacity planning

4.8 Economic Domain

The Economic domain manages all financial transactions and value exchange within the platform.

4.8.1 Billing Service (port 8050)

Invoicing and billing:

- Invoice generation and management
- Usage-based billing calculations
- Invoice lifecycle: DRAFT → SENT → PAID → OVERDUE → CANCELLED

4.8.2 Payment Service (port 8090)

Payment processing:

- Stripe integration for international payments
- Paymob Accept for Egyptian cards and Vodafone Cash
- Payment lifecycle: PENDING → PROCESSING → COMPLETED → FAILED → REFUNDED
- Webhook handling for asynchronous payment confirmations

4.8.3 Subscription Service

Subscription management:

- Tiered subscription plans (Free, Pro, Enterprise)
- Automated billing cycles
- Subscription upgrades, downgrades, and cancellations

4.8.4 Coin Service

MASAREN Coin (MRC) management:

- MRC token creation and distribution
- MRC-to-EGP exchange rate management
- Wallet balance tracking per user
- Transaction history and audit trails

4.9 Additional Services

4.9.1 Notification Service (port 8060)

Multi-channel notification delivery:

- Firebase Cloud Messaging (FCM) for push notifications
- Email notifications via SMTP
- In-app notification center
- Notification preferences and opt-out management
- Template-based notification rendering

4.9.2 Chat Service

Real-time communication:

- Direct messaging between users
- Group chats for program teams
- NATS-powered real-time updates
- Message persistence and history
- Read receipts and typing indicators

4.9.3 Reporting Domain

- **Report Service:** Report generation with configurable templates
- **Analytics Service:** Live SQL analytics with query builder
- **Forecast Service:** Predictive analytics and trend forecasting

4.9.4 Partner Service

Partner organization management:

- Partner onboarding and verification
- Partnership agreement management
- Co-branded program support

4.9.5 Ads Service

Advertising and brokerage:

- Targeted advertising within the platform
- Opportunity promotion and featured listings
- Brokerage fee management

4.9.6 Platform Service

Platform-level aggregation and revenue management:

- Cross-service revenue aggregation
- Platform fee calculation and collection
- System-wide analytics and reporting

5 Combined Backend Architecture

5.1 Purpose and Design

The Combined Backend is a design pattern unique to the MASAREN platform that merges all 35+ microservices into a single FastAPI process for local development efficiency. This approach eliminates the overhead of running 30+ separate Docker containers during development while maintaining complete service compatibility.

Purpose: Single-process backend that merges all microservices for local development

Architecture: Imports each service as a uniquely-named package and routes requests to appropriate handlers

Benefits: Local development on a laptop or small cloud droplet without running the full Docker stack

Complexity: Duplicate table registration requires SQLAlchemy Table.___new___ patching

Performance: Single process eliminates network overhead between services

5.2 Service Registration Mechanism

The combined backend programmatically registers each service package at runtime. Each service is given a unique alias and its source directory is added to Python's module path:

```
SERVICES = [
    ("ms_identity",      "identity/identity-service"),
    ("ms_auth",          "identity/authentication-service"),
    ("ms_authz",         "identity/authorization-service"),
    ("ms_organization",  "execution/organization-service"),
    ("ms_program",       "execution/program-service"),
    ("ms_task",          "execution/task-service"),
    ("ms_workflow",      "execution/workflow-service"),
    ("ms_verification",  "verification/verification-service"),
    ("ms_evidence",      "verification/evidence-service"),
    ("ms_gps",           "verification/gps-service"),
    ("ms_audit",         "verification/audit-service"),
    ("ms_media",         "verification/media-service"),
    ("ms_submission",    "verification/submission-service"),
    ("ms_graph",         "graph/graph-service"),
    ("ms_reputation",    "reputation/portfolio-service"),
    ("ms_impact_score",  "reputation/impact-score-service"),
    ("ms_org_score",     "reputation/organization-score-service"),
    ("ms_marketplace",   "marketplace/opportunity-service"),
    ("ms_matching",      "marketplace/matching-service"),
    ("ms_report",        "reporting/report-service"),
    ("ms_analytics",     "reporting/analytics-service"),
    ("ms_forecast",      "reporting/forecast-service"),
    ("ms_notification",  "notifications/notification-service"),
    ("ms_chat",          "chat-service"),
```

```
    ("ms_billing",       "billing/billing-service"),
    ("ms_payment",       "billing/payment-service"),
    ("ms_subscription",  "billing/subscription-service"),
    ("ms_partner",       "partners/partner-service"),
    ("ms_coin",          "coin/coin-service"),
    ("ms_ads",           "ads/ads-service"),
    ("ms_platform",      "platform/platform-service"),
]
```

Listing 1: Service Registration in combined/main.py

5.3 Package Registration Function

Each service is dynamically registered as a top-level Python package. This allows standard Python imports to resolve service modules correctly:

```
def _register_service_package(alias: str, rel: str) -> None:
    \"\"\"Register␣backend/<rel>␣as␣a␣top-level␣package␣named␣alias.\"\"\"
␣␣␣␣svc_root␣=␣os.path.join(BACKEND,␣rel)
␣␣␣␣pkg␣=␣types.ModuleType(alias)
␣␣␣␣pkg.__path__␣=␣[svc_root]
␣␣␣␣pkg.__package__␣=␣alias
␣␣␣␣sys.modules[alias]␣=␣pkg
```

Listing 2: Dynamic Package Registration

5.4 Duplicate Table Resolution

A critical challenge of merging 30+ independent services is that many define the same SQLAlchemy ORM tables (e.g., `users`, `organizations`). The solution is a monkey-patch on `Table.__new__` that deduplicates table registrations:

```
import sqlalchemy as _sa
from sqlalchemy import Table as _Table

_orig_table_new = _Table.__new__

def _patched_table_new(cls, *args, **kwargs):
    metadata = kwargs.get("metadata") or \
                (args[1] if len(args) >= 2 else None)
    name = kwargs.get("name") or \
           (args[0] if args else None)
    schema = kwargs.get("schema")

    if metadata is not None and name is not None:
        key = (schema, name) if schema else name
        if key in metadata.tables:
            return metadata.tables[key]

    return _orig_table_new(cls, *args, **kwargs)

_Table.__new__ = staticmethod(_patched_table_new)
```

Listing 3: SQLAlchemy Table Deduplication

5.5 Route Merging

After registering packages and patching tables, the combined backend iterates through all services and merges their FastAPI routes into a single application:

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI(
    title="MASAREN␣Combined␣Backend",
    version="0.1.0"
)

for alias, rel in SERVICES:
    try:
        _register_service_package(alias, rel)
        mod = importlib.import_module(f"{alias}.src.main")
        svc = getattr(mod, "app", None)

        if svc is None:
            continue

        for r in list(svc.routes):
            if getattr(r, "path", None) in SKIP:
                svc.routes.remove(r)

        app.include_router(svc.router)

        for exc_type, handler in \
                svc.exception_handlers.items():
            if exc_type not in app.exception_handlers:
                app.add_exception_handler(exc_type, handler)

        _included.append(alias)
        print(f"[combined]␣OK␣included␣{alias}", flush=True)
    except Exception as e:
        print(f"[combined]␣FAIL␣{alias}:␣{e!r}", flush=True)

app.add_middleware(
    CORSMiddleware,
    allow_origins=os.getenv(
        "CORS_ORIGINS",
        "http://localhost:3000,http://localhost:3001"
    ).split(","),
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Listing 4: Route Merging

5.6 Service Skip List

Certain routes are excluded from the combined backend to avoid conflicts:

- Gateway routes (`/api/v1/*`) – the gateway should not proxy to itself
- Duplicate health check endpoints (only one `/health` is kept)
- OpenAPI documentation endpoints (only the combined app's docs are exposed)

6 Web Application Architecture

6.1 Component Architecture

The web application at `apps/web` is a React 18.2 + TypeScript application built with Vite 5.1. It follows a component-based architecture with clear separation of concerns:

6.1.1 Directory Structure

```
apps/web/src/
  components/
    layout/          TopNav, Sidebar, Header, RightRail
    common/          Branding, ErrorBoundary, Icons
  pages/             30+ page components
  services/          API client layer
  store/             React Context providers
  hooks/             Custom React hooks
  types/             TypeScript type definitions
  test/              Test utilities and setup
  i18n.tsx           Internationalization configuration
```

6.1.2 Page Structure

The application defines 30+ pages accessible through React Router v6:

Page	Route	Description
LandingPage	/, /landing	Public landing with platform overview
LoginPage	/login	Authentication with email/password and Google OAuth
RegisterPage	/register	User registration with email verification
DashboardPage	/dashboard	Personal dashboard with metrics and activity
ProgramsPage	/programs	Program discovery and enrollment
TasksPage	/tasks	Task management and assignment
EvidencePage	/evidence	Evidence submission and tracking
PassportPage	/passport	Digital impact passport
OpportunitiesPage	/opportunities	Marketplace opportunity discovery
PortfolioPage	/portfolio	Impact asset portfolio
ProfilePage	/profile	User profile management
PersonnelProfilePage	/personnel/:id	Public personnel profile
InstitutionProfilePage	/institution/:id	Institution profile view

Page	Route	Description
AdminPage	/admin	Administrative dashboard (ADMIN/SUPER_ADMIN)
NotificationsPage	/notifications	Notification center
ReportsPage	/reports	Analytics and reports
BillingPage	/billing	Billing and subscription management
ChatPage	/chat	Real-time messaging
FeedPage	/feed	Social activity feed
WalletPage	/wallet	MRC wallet management
...	...	Additional pages for ads, payments, revenue, challenges, etc.

6.2 Routing Architecture

The application implements protected routes with role-based access control:

```
function RequireRole({
  roles,
  children
}: {
  roles: string[];
  children: ReactElement;
}) {
  const { user, isAuthenticated } = useAuth();
  if (!isAuthenticated) return <Navigate to="/login" replace />;
  if (user && !roles.includes(user.role))
    return <Navigate to="/dashboard" replace />;
  return children;
}
```

Listing 5: Protected Route Implementation

6.3 State Management

The application uses a dual state management approach:

React Context Client-side state for authentication (`AuthProvider`), theme preferences, and UI state

TanStack Query Server state management with automatic caching, background refetching, and optimistic updates

6.4 Internationalization

The application supports bilingual English/Arabic operation:

```
// apps/web/src/i18n.tsx
import i18n from 'i18next';
```

```
import { initReactI18next } from 'react-i18next';

i18n.use(initReactI18next).init({
  resources: {
    en: { translation: enTranslations },
    ar: { translation: arTranslations }
  },
  lng: 'en',
  fallbackLng: 'en',
  interpolation: { escapeValue: false }
});
```

Listing 6: i18n Configuration

6.5 Authentication Flow

1. User navigates to login page
2. User authenticates via email/password or Google OAuth popup
3. Backend issues JWT access token and refresh token
4. Axios interceptor attaches JWT to all API requests
5. On 401 response, interceptor attempts token refresh
6. On refresh failure, user is redirected to login

6.6 API Client Layer

The services layer provides type-safe API interaction:

```
// apps/web/src/services/
import axios from 'axios';

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL ||
           'http://localhost:8080/api/v1',
});

api.interceptors.request.use((config) => {
  const token = getAccessToken();
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

api.interceptors.response.use(
  (response) => response,
  async (error) => {
    if (error.response?.status === 401) {
      await refreshToken();
```

```
      return api.request(error.config);
    }
    return Promise.reject(error);
  }
);
```

Listing 7: API Client Structure

7 Mobile Application Architecture

7.1 Architecture Overview

The mobile application at `apps/mobile` is built with Flutter 3.22+ and follows the MVVM (Model-View-ViewModel) architecture pattern with Provider for state management. The app provides the same feature set as the web application, optimized for mobile form factors with offline support and device-native capabilities.

7.2 Directory Structure

```
apps/mobile/lib/
  features/            Feature-specific modules
  providers/           ChangeNotifier providers
  services/            API client and service layer
  widgets/             Reusable UI components
  models/              Data models and DTOs
  screens/             Full-page screen widgets
  utils/               Utility functions and helpers
  main.dart            Application entry point
```

7.3 Screen Structure

The Flutter app implements the following screens:

- Login and Registration screens with Google OAuth integration
- Dashboard with activity summary and quick actions
- Program discovery and enrollment screens
- Task list and detail screens with mobile-optimized forms
- Evidence capture screen with camera integration and GPS binding
- Digital passport visualization
- Opportunity marketplace with search and filters
- Impact portfolio with charts and metrics
- Profile management with edit capabilities
- Real-time chat interface
- Social feed with infinite scroll
- Notification center with push notification handling
- Billing and subscription management
- Wallet for MRC balance and transactions

7.4 State Management

The app uses the Provider package for state management:

```
// main.dart
void main() {
  runApp(
    MultiProvider(
      providers: [
        ChangeNotifierProvider(create: (_) => AuthProvider()),
        ChangeNotifierProvider(create: (_) => ThemeProvider()),
        ChangeNotifierProvider(create: (_) => ProgramProvider()),
        ChangeNotifierProvider(create: (_) => TaskProvider()),
        ChangeNotifierProvider(create: (_) => EvidenceProvider()),
        ChangeNotifierProvider(create: (_) => NotificationProvider()),
      ],
      child: const MasarenApp(),
    ),
  );
}
```

Listing 8: Provider Setup

7.5 Navigation Architecture

The app uses Flutter Navigator with a custom bottom navigation bar:

- **Bottom Navigation Tabs:** Dashboard, Programs, Tasks, Profile, More
- **Drawer Menu:** Settings, Help, About, Logout
- **Deep Linking:** Notification-driven navigation to specific screens
- **Modal Routes:** Evidence capture, chat, and form modals

7.6 Mobile-Specific Features

The mobile app leverages device capabilities not available on the web:

- **Camera Integration:** Direct evidence capture through device camera
- **GPS Location Binding:** Automatic coordinates attachment to submissions
- **Offline Mode:** Queue submissions when offline, sync when connected
- **Push Notifications:** FCM-based real-time alerts for assignments and verification
- **Biometric Auth:** Fingerprint/Face ID for quick app unlock
- **Image Gallery:** Photo library access for evidence selection
- **File Management:** Download and open impact certificates and reports

7.7 API Integration

```
class ApiService {
  static const String baseUrl =
    'http://localhost:8080/api/v1';

  Future<Map<String, dynamic>> get(
    String endpoint, {
    Map<String, String>? headers,
  }) async {
    final response = await http.get(
      Uri.parse('$baseUrl/$endpoint'),
      headers: {
        'Content-Type': 'application/json',
        'Authorization': 'Bearer ${await getToken()}',
        ...?headers,
      },
    );
    return json.decode(response.body);
  }

  Future<Map<String, dynamic>> post(
    String endpoint, {
    Map<String, dynamic>? body,
  }) async {
    final response = await http.post(
      Uri.parse('$baseUrl/$endpoint'),
      headers: {
        'Content-Type': 'application/json',
        'Authorization': 'Bearer ${await getToken()}',
      },
      body: json.encode(body),
    );
    return json.decode(response.body);
  }
}
```

Listing 9: API Service Layer

7.8 Bilingual Support

The mobile app provides full bilingual support similar to the web:

```
class MasarenLocalizations {
  static const Map<String, Map<String, String>> _localized = {
    'en': {
      'app.title': 'MASAREN',
      'nav.home': 'Home',
      'nav.programs': 'Programs',
      'nav.tasks': 'Tasks',
      'nav.profile': 'Profile',
    },
```

```
    'ar': {
      'app.title': '',
      'nav.home': '',
      'nav.programs': '',
      'nav.tasks': '',
      'nav.profile': ' ',
    },
  };
}
```

Listing 10: Internationalization Setup

8 Event-Driven Architecture

8.1 Kafka Event Bus

Apache Kafka 3.7 serves as the central nervous system of the MASAREN platform, providing durable, scalable, and fault-tolerant event streaming. Kafka runs in KRaft mode (no Zookeeper dependency) for simplified operations.

8.2 Event Catalog

The platform defines 20+ domain events organized by bounded context:

Event	Domain	Description
organization_created	Execution	A new organization has been registered
program_created	Execution	A new program has been launched
task_created	Execution	A new task has been defined
task_assigned	Execution	Task assigned to a volunteer
task_started	Execution	Volunteer began task execution
task_completed	Execution	Volunteer marked task as done
location_reached	Verification	GPS confirms arrival at target location
submission_uploaded	Verification	Evidence submitted for verification
evidence_uploaded	Verification	Media evidence uploaded
evidence_verified	AI/Vision	AI completed verification analysis
fraud_detected	AI/Fraud	Suspicious pattern identified
impact_asset_created	AI/Economist	New impact asset minted
impact_score_updated	Reputation	PMS recalculated
organization_score_updated	Reputation	OII recalculated
portfolio_updated	Reputation	User portfolio changed
graph_updated	Graph	Neo4j knowledge graph updated

8.3 Event Schema

Each domain event follows a standardized schema:

```
from pydantic import BaseModel
from datetime import datetime
from uuid import uuid4

class DomainEvent(BaseModel):
    event_id: str = str(uuid4())
    event_type: str
    aggregate_id: str
    aggregate_type: str
    event_version: int = 1
    timestamp: datetime = datetime.utcnow()
```

```
    data: dict
    metadata: dict = {}
```

Listing 11: Domain Event Base Class

8.4 Transactional Outbox Pattern

To ensure atomicity between database operations and event publication, every service uses the Transactional Outbox pattern:

```
from sqlalchemy import Column, String, JSON, DateTime, Enum
from sqlalchemy.ext.declarative import declarative_base

class OutboxMessage(Base):
    __tablename__ = "outbox_messages"

    id = Column(String, primary_key=True, default=uuid4)
    event_type = Column(String, nullable=False)
    aggregate_id = Column(String, nullable=False)
    aggregate_type = Column(String, nullable=False)
    payload = Column(JSON, nullable=False)
    status = Column(
        Enum("PENDING", "PUBLISHED", "FAILED"),
        default="PENDING"
    )
    created_at = Column(DateTime, default=datetime.utcnow)
    published_at = Column(DateTime, nullable=True)

def publish_event(event: DomainEvent) -> None:
    session = Session()
    outbox = OutboxMessage(
        event_type=event.event_type,
        aggregate_id=event.aggregate_id,
        aggregate_type=event.aggregate_type,
        payload=event.dict()
    )
    session.add(outbox)
    session.commit()
```

Listing 12: Transactional Outbox Implementation

8.5 Event Processing Pipeline

1. Domain action triggers `publish_event()` within a database transaction
2. Event is saved to the `outbox_messages` table in the same transaction
3. Background processor polls for `PENDING` outbox messages
4. Each pending message is serialized and published to the appropriate Kafka topic
5. On successful publication, the message status is updated to `PUBLISHED`

6. Downstream services consume events with at-least-once delivery guarantees

7. Idempotent consumers deduplicate events using `event_id`

8.6 Consumer Implementation

```python
from shared.utils.kafka_consumer import KafkaConsumer

class VerificationEventConsumer(KafkaConsumer):
    def __init__(self):
        super().__init__(
            topic="evidence_verified",
            group_id="verification-service-group"
        )

    async def process(self, event: DomainEvent) -> None:
        if await self._is_duplicate(event.event_id):
            return
        if event.data.get("status") == "APPROVED":
            await self._trigger_economist_agent(event)
        await self._mark_processed(event.event_id)

    async def _is_duplicate(self, event_id: str) -> bool:
        return await redis_client.sismember(
            "processed_events", event_id
        )

    async def _mark_processed(self, event_id: str) -> None:
        await redis_client.sadd("processed_events", event_id)
```

Listing 13: Idempotent Kafka Consumer

9 Shared Kernel

The shared kernel at `backend/shared` provides reusable infrastructure code imported by all services as `shared.*`. It eliminates code duplication and ensures consistent patterns across the entire platform.

9.1 Kernel Modules

Module	Purpose
`events/`	Domain event base classes and all 20+ event contract definitions
`utils/cqrs.py`	CommandBus and QueryBus with middleware support for cross-cutting concerns
`utils/kafka_producer.py`	Idempotent Kafka publisher with retry logic and error handling
`utils/kafka_consumer.py`	At-least-once consumer base with automatic offset management
`utils/outbox.py`	Transactional Outbox pattern implementation
`utils/cloud_llm_client.py`	Unified abstraction for OpenAI and Google AI client APIs
`utils/app_factory.py`	Shared FastAPI app factory with standardized configuration
`utils/health.py`	Composable health check system with Redis, PostgreSQL, Kafka checks
`utils/logging.py`	Structured JSON logging with request ID propagation
`middleware/`	JWT validation, RBAC enforcement, bcrypt password hashing
`base_entity.py`	Abstract base for all domain entities with common fields
`base_repository.py`	Generic CRUD repository with filtering and pagination
`constants.py`	Platform-wide constants, enums, and configuration
`dtos.py`	Shared Data Transfer Objects for cross-service communication
`exceptions.py`	Standardized exception hierarchy with HTTP mapping
`testing/`	Test fixtures, factories, and utilities
`types.py`	Custom type definitions and generic aliases

9.2 CQRS Pattern

```
from abc import ABC, abstractmethod
```

```python
from dataclasses import dataclass

class Command(ABC):
    pass

class Query(ABC):
    pass

class CommandBus:
    def __init__(self):
        self._handlers = {}

    def register(self, command_type, handler):
        self._handlers[command_type] = handler

    async def dispatch(self, command: Command):
        handler = self._handlers.get(type(command))
        if not handler:
            raise ValueError(
                f"No␣handler␣for␣{type(command).__name__}"
            )
        return await handler.handle(command)

class QueryBus:
    def __init__(self):
        self._handlers = {}

    def register(self, query_type, handler):
        self._handlers[query_type] = handler

    async def ask(self, query: Query):
        handler = self._handlers.get(type(query))
        if not handler:
            raise ValueError(
                f"No␣handler␣for␣{type(query).__name__}"
            )
        return await handler.handle(query)
```

Listing 14: Command and Query Bus

9.3 Health Check System

```python
from shared.utils.health import (
    HealthCheck,
    RedisHealthCheck,
    PostgresHealthCheck,
    KafkaHealthCheck,
)

async def health_check():
    checks = HealthCheck([
        PostgresHealthCheck(),
```

```
        RedisHealthCheck(),
        KafkaHealthCheck(),
    ])
    return await checks.run()
```

Listing 15: Health Check Implementation

10 Infrastructure and Deployment

10.1 Docker Compose Architecture

The `docker-compose.yml` file defines 30+ services and infrastructure components for full-stack local development:

Component	Image	Port	Purpose
Gateway	backend/Dockerfile.service	8000	BFF API Gateway
Web Frontend	apps/web/Dockerfile	3000	React web application
Admin Frontend	apps/admin/Dockerfile	3001	Admin panel
Identity	backend/Dockerfile.service	8001	Identity management
Auth	backend/Dockerfile.service	8002	Authentication
Authz	backend/Dockerfile.service	8003	Authorization
Organization	backend/Dockerfile.service	8004	Organization management
Program	backend/Dockerfile.service	8005	Program management
Task	backend/Dockerfile.service	8006	Task management
Workflow	backend/Dockerfile.service	8007	Workflow orchestration
Verification	backend/Dockerfile.service	8008	Verification engine
Evidence	backend/Dockerfile.service	8009	Evidence management
PostgreSQL	postgis/postgis:16-3.4	5432	Primary database
Neo4j	neo4j:5-community	7687	Knowledge graph
Redis	redis:7-alpine	6379	Cache and sessions
Kafka	apache/kafka:3.7.2	9092	Event bus
MinIO	minio/minio	9000	Blob storage
NATS	nats:latest	4222	Real-time messaging

10.2 Dockerfile Pattern

```
FROM python:3.11-slim

WORKDIR /app

RUN apt-get update && apt-get install -y gcc libpq-dev \\
    && rm -rf /var/lib/apt/lists/*

COPY backend/shared /app/shared

ARG SERVICE_PATH
COPY \${SERVICE_PATH} /app/service
WORKDIR /app/service

RUN pip install --no-cache-dir -r requirements.txt
```

```
EXPOSE 8000

CMD ["uvicorn", "src.main:app", \\
     "--host", "0.0.0.0", "--port", "8000"]
```

Listing 16: Dockerfile.service Template

10.3 Database Migrations

```
CREATE EXTENSION IF NOT EXISTS postgis;

CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    display_name VARCHAR(255),
    role VARCHAR(50) DEFAULT 'VOLUNTEER',
    email_verified BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE organizations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(255) NOT NULL,
    description TEXT,
    logo_url TEXT,
    created_at TIMESTAMPTZ DEFAULT NOW()
);
```

Listing 17: Migration Example (V001__initial.sql)

10.4 Kubernetes Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: identity-service
spec:
  replicas: 3
  selector:
    matchLabels:
      app: identity-service
  template:
    metadata:
      labels:
        app: identity-service
    spec:
      containers:
      - name: identity
```

```
        image: masaren/identity-service:latest
        ports:
        - containerPort: 8000
        livenessProbe:
          httpGet:
            path: /health
            port: 8000
        readinessProbe:
          httpGet:
            path: /health/ready
            port: 8000
```

Listing 18: Service Deployment Template

10.5 CI/CD Pipeline

```
name: MASAREN CI

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  test-shared:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: |
          cd backend/shared
          pip install -r requirements.txt
          pytest --cov=shared --cov-report=xml

  test-services:
    strategy:
      matrix:
        service: [identity, auth, organization, program, task]
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: |
          cd backend/\${{ matrix.service }}
          pip install -r requirements.txt
          pytest

  build-docker:
    needs: [test-shared, test-services]
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: |
```

```
          docker build -t masaren/app:latest .
          docker push masaren/app:latest

  deploy:
    needs: [build-docker]
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'
    steps:
      - run: |
          kubectl set image deployment/app app=masaren/app:\${{ github.sha }}
```

Listing 19: CI Pipeline Structure

11 Testing Strategy

The MASAREN platform employs a comprehensive testing strategy targeting 90%+ code coverage across all services.

11.1 Testing Pyramid

Unit Tests: Test individual functions, commands, queries, and domain logic in isolation. Mock all external dependencies (database, Kafka, Redis, AI APIs).

Integration Tests: Test service interactions with real database instances, Kafka topics, and Redis. Verify the Transactional Outbox pattern end-to-end.

Contract Tests: Verify API contract compliance between services using OpenAPI schema validation.

Event Tests: Verify event schemas, consumer idempotency, and event replay capabilities.

E2E Tests: Full-stack tests using Playwright (web) and Maestro (mobile) covering critical user journeys.

11.2 Test Configuration

```
import pytest
from shared.testing import (
    DatabaseFixture,
    KafkaFixture,
    RedisFixture,
    AuthFixture,
)

@pytest.fixture
async def db():
    async with DatabaseFixture() as fixture:
        await fixture.create_tables()
        yield fixture
        await fixture.drop_tables()

@pytest.fixture
async def kafka():
    async with KafkaFixture() as fixture:
        yield fixture

@pytest.fixture
def auth_token():
    return AuthFixture().generate_token(
        user_id="test-user",
        roles=["VOLUNTEER"]
    )
```

Listing 20: Test Fixtures

11.3 Example Unit Test

```python
import pytest
from datetime import datetime
from uuid import uuid4

class TestTaskService:
    async def test_create_task_success(self, db, auth_token):
        service = TaskService(db.session)
        command = CreateTaskCommand(
            program_id=str(uuid4()),
            title="Plant␣50␣trees",
            description="Reforestation␣task",
            location={"lat": 30.0, "lng": 31.0},
            deadline=datetime(2026, 12, 31),
        )
        result = await service.create_task(command, auth_token)
        assert result.status == "CREATED"
        assert result.title == "Plant␣50␣trees"

    async def test_create_task_missing_title(self, db, auth_token):
        with pytest.raises(ValidationError):
            command = CreateTaskCommand(
                program_id=str(uuid4()),
                title="",
                description="",
            )
            await service.create_task(command, auth_token)
```

Listing 21: Unit Test Example

11.4 E2E Testing with Playwright

```javascript
import { test, expect } from '@playwright/test';

test('volunteer can complete task flow', async ({ page }) => {
  await page.goto('/login');
  await page.fill('[name=email]', 'volunteer@test.com');
  await page.fill('[name=password]', 'password123');
  await page.click('button[type=submit]');
  await expect(page).toHaveURL('/dashboard');

  await page.click('text=Tasks');
  await expect(page).toHaveURL('/tasks');

  await page.click('text=Plant 50 trees');
  await page.click('text=Accept Task');

  await page.click('text=Submit Evidence');
  await page.setInputFiles('input[type=file]', 'test-assets/trees.jpg');
  await page.click('text=Submit');
```

```
  await expect(page.locator('text=Evidence Submitted')).toBeVisible();
});
```

Listing 22: Web E2E Test

11.5 Mobile E2E Testing with Maestro

```
appId: com.masaren.app
---
- launchApp
- tapOn: "Login"
- inputText: "volunteer@test.com"
- tapOn: "Next"
- inputText: "password123"
- tapOn: "Sign In"
- assertVisible: "Dashboard"
- tapOn: "Tasks"
- assertVisible: "Plant 50 trees"
- tapOn: "Accept Task"
- tapOn: "Capture Evidence"
- takeScreenshot: "evidence_capture"
- tapOn: "Submit"
- assertVisible: "Evidence Submitted"
```

Listing 23: Mobile E2E Flow

12 Security Architecture

12.1 Authentication and Authorization

The platform implements a defense-in-depth security model:

- **JWT-Based Authentication:** RS256 signed JWT tokens with short expiration (15 minutes) and refresh token rotation (7 days)
- **RBAC:** Four roles with hierarchical permissions: VOLUNTEER < ORG_ADMIN < ADMIN < SUPER_ADMIN
- **Google OAuth:** Popup-based authentication with PKCE flow for enhanced security
- **API Key Authentication:** Service-to-service communication uses API keys stored in environment variables
- **Rate Limiting:** Redis-backed rate limiting per IP and user ID
- **CORS:** Strict origin validation with whitelisted domains
- **Input Validation:** Pydantic models validate all API inputs with comprehensive error messages
- **SQL Injection Prevention:** SQLAlchemy ORM with parameterized queries throughout

12.2 Token Management

```
from datetime import datetime, timedelta
import jwt

class TokenManager:
    def __init__(self, secret: str):
        self.secret = secret
        self.access_expiry = timedelta(minutes=15)
        self.refresh_expiry = timedelta(days=7)

    def create_access_token(self, user_id: str, roles: list[str]) -> str:
        payload = {
            "sub": user_id,
            "roles": roles,
            "type": "access",
            "iat": datetime.utcnow(),
            "exp": datetime.utcnow() + self.access_expiry,
        }
        return jwt.encode(payload, self.secret, algorithm="HS256")

    def create_refresh_token(self, user_id: str) -> str:
        payload = {
            "sub": user_id,
            "type": "refresh",
```

```
            "iat": datetime.utcnow(),
            "exp": datetime.utcnow() + self.refresh_expiry,
        }
        return jwt.encode(payload, self.secret, algorithm="HS256")

    def verify_token(self, token: str) -> dict:
        try:
            return jwt.decode(token, self.secret, algorithms=["HS256"])
        except jwt.ExpiredSignatureError:
            raise AuthenticationError("Token␣expired")
        except jwt.InvalidTokenError:
            raise AuthenticationError("Invalid␣token")
```

Listing 24: JWT Token Management

12.3 RBAC Middleware

```
from fastapi import Request, HTTPException
from functools import wraps

def require_roles(allowed_roles: list[str]):
    def decorator(func):
        @wraps(func)
        async def wrapper(request: Request, *args, **kwargs):
            user = request.state.user
            if not user:
                raise HTTPException(status_code=401, detail="Authentication␣
                    required")
            if not any(role in user.get("roles", []) for role in allowed_roles):
                raise HTTPException(status_code=403, detail="Insufficient␣
                    permissions")
            return await func(request, *args, **kwargs)
        return wrapper
    return decorator
```

Listing 25: Role-Based Access Control Middleware

12.4 Data Privacy

- **GDPR Compliance:** User data export, deletion, and consent management APIs
- **Data Encryption:** All data encrypted at rest and in transit (TLS 1.3)
- **PII Handling:** Personally Identifiable Information stored separately with restricted access
- **Audit Trails:** All access to sensitive data is logged with user ID, timestamp, and action
- **Evidence Privacy:** Evidence media is access-controlled with per-program visibility rules

12.5 Security Best Practices

- Secrets stored in environment variables, never in code
- `JWT_SECRET` must be at least 256 bits and unique per deployment
- Database passwords generated with cryptographic random generators
- All API endpoints behind the Gateway – no direct service exposure
- Health endpoints are read-only and expose no sensitive data
- Rate limiting prevents brute force and DDoS attacks
- Input validation at API boundaries prevents injection attacks
- CORS configured with explicit allowed origins

13 Environment Configuration

13.1 Required Environment Variables

Variable	Required	Description
JWT_SECRET	Yes	Shared secret for JWT signing (min 256-bit)
DATABASE_URL	Yes	PostgreSQL connection string
REDIS_URL	Yes	Redis connection string
KAFKA_BOOTSTRAP_SERVERS	Yes	Kafka broker addresses
NEO4J_URL	Yes	Neo4j connection string
OPENAI_API_KEY	Yes	OpenAI API key for AI agents
PUBLIC_GATEWAY_URL	Yes	Public API Gateway URL
STRIPE_SECRET_KEY	No	Stripe payment processing
STRIPE_WEBHOOK_SECRET	No	Stripe webhook verification
PAYMOB_API_KEY	No	Paymob payment processing
PAYMOB_INTEGRATION_ID	No	Paymob integration identifier
FCM_SERVICE_ACCOUNT_JSON	No	Firebase Cloud Messaging
MRC_PER_EGP	No	MASAREN Coin exchange rate
CREWAI_BROKER_URL	No	AI orchestrator broker URL
CORS_ORIGINS	No	Allowed CORS origins

13.2 Environment Configuration File

```
JWT_SECRET=your-256-bit-secret-here
DATABASE_URL=postgresql://masaren:password@localhost:5432/masaren
REDIS_URL=redis://localhost:6379/0
KAFKA_BOOTSTRAP_SERVERS=localhost:9092
NEO4J_URL=bolt://neo4j:password@localhost:7687
OPENAI_API_KEY=sk-your-openai-key
PUBLIC_GATEWAY_URL=http://localhost:8080/api/v1
```

Listing 26: .env.example

14 Development Workflow

14.1 Local Development Setup

1. Clone the repository and copy environment configuration
2. Start infrastructure dependencies with Docker Compose
3. Run individual services natively using `uvicorn`
4. Or launch the combined backend for all-in-one development
5. Frontend development with Vite HMR and Flutter hot reload

```
cp .env.example .env
docker compose up -d postgres neo4j redis kafka minio
cd backend/identity/identity-service
pip install -r requirements.txt
uvicorn src.main:app --reload --port 8001
cd apps/web && npm install && npm run dev
cd apps/mobile && flutter pub get && flutter run
```

Listing 27: Development Setup Commands

14.2 Native Development Script

```
gateway       :8080
identity      :8001
auth          :8002
organization  :8004
program       :8005
task          :8006
workflow      :8007
verification  :8008
evidence      :8009
graph         :8011
opportunity   :8013
matching      :8014
billing       :8050
notification  :8060
orchestrator  :8070
fraud-python  :8090
auth-python   :8093
impact-score  :8094
```

Listing 28: Native Port Assignments

14.3 Engineering Rules

- **Never** bypass domain events – always use Transactional Outbox

- **Never** couple services to another service's database
- **Never** call another service's database directly
- **Never** run AI inference locally – cloud only
- **Never** put business logic in controllers
- **Never** bypass CommandBus/QueryBus
- **Never** publish Kafka events without the Outbox pattern
- **Never** create circular service dependencies
- **Always** use the Repository pattern for data access
- **Always** validate DTOs with Pydantic
- **Always** publish domain events for state changes
- **Always** write unit + integration + contract tests
- **Always** maintain independent deployability
- **Always** use structured JSON logging
- **Always** expose health check endpoints
- **Always** implement idempotent event consumers
- **Always** keep domain models framework-independent

14.4 Development Stages Completed

Stage	Description	Status
0	Project specification	Completed
1	Monorepo structure (423 directories, 24+ services)	Completed
2	Shared kernel (22 modules)	Completed
3	Identity domain (54 files, 30+ tests)	Completed
4	Execution domain (organization, program, task, workflow)	Completed
5	Verification domain (6 services)	Completed
6	AI agents (full implementation)	Completed

Stage	Description	Status
7	Reputation (PMS, OII, portfolio)	Completed
8	Marketplace (opportunity, matching)	Completed
9	Reporting and analytics	Completed
10	Knowledge graph (Neo4j)	Completed
11	Frontend (Flutter + React)	Completed
12	Production infrastructure (Docker, K8s, CI/CD)	Completed

15 Known Gaps and Future Work

15.1 Current Gaps

- Per-function dead-code analysis (ruff/vulture) across all 30+ services
- Several AI agents implemented but not wired into docker-compose.yml
- OpenSearch search layer planned but not present in compose stack
- Go/Rust targets aspirational only; current implementation is Python
- gRPC remains aspirational; REST + Kafka is the real transport

15.2 Future Roadmap

1. **OpenSearch Integration:** Full-text search across programs, opportunities, and impact assets
2. **AI/ML Enhancement:** Additional AI agents for predictive analytics and recommendations
3. **Event Sourcing:** Complete event sourcing for core domain aggregates
4. **Saga Orchestration:** Distributed transaction management across bounded contexts
5. **Performance Optimization:** Query optimization, caching strategies, CDN integration
6. **Mobile Offline Mode:** Enhanced offline support with conflict resolution
7. **Advanced Analytics:** Real-time dashboards with drill-down capabilities
8. **Multi-Tenancy:** Tenant isolation for enterprise deployments
9. **API Versioning:** Formal API versioning strategy for backward compatibility

16 Appendices

16.1 Appendix A: Service Port Map

Service	Port	Docker Compose Key
API Gateway	8080	gateway
Identity Service	8001	identity
Authentication Service	8002	auth
Authorization Service	8003	authz
Organization Service	8004	organization
Program Service	8005	program
Task Service	8006	task
Workflow Service	8007	workflow
Verification Service	8008	verification
Evidence Service	8009	evidence
Graph Service	8011	graph
Opportunity Service	8013	marketplace
Matching Service	8014	matching
Billing Service	8050	billing
Notification Service	8060	notification
AI Orchestrator	8070	orchestrator
Fraud Detection (Python)	8090	fraud-python
Auth (Python)	8093	auth-python
Impact Score (Python)	8094	impact-score-python
Web Frontend	3000	web
Admin Frontend	3001	admin

16.2 Appendix B: Repository Layout

```
MASAREN/
  README.md
  PLATFORM_GENOME.md
  platform-genome.json
  docker-compose.yml
  .env.example
  apps/
    web/                    React + TypeScript (port 3000)
    admin/                  React + TypeScript (Mantine, port 3001)
    mobile/                 Flutter (Android/iOS)
  backend/
    shared/                 Shared kernel package
```

```
    combined/main.py        All models loader
    identity/               Identity domain (3 services)
    execution/              Execution domain (4 services)
    verification/           Verification domain (6 services)
    ai/                     AI agents + orchestrator
    graph/                  Graph service
    reputation/             Reputation domain (3 services)
    marketplace/            Marketplace domain (2 services)
    reporting/              Reporting domain (3 services)
    notifications/          Notification service
    billing/                Billing domain (3 services)
    partners/               Partner service
    coin/                   Coin service
    ads/                    Ads service
    platform/               Platform service
    gateway/                API Gateway
    chat-service/           Chat service
  docs/                     Documentation
  infrastructure/           DB migrations, K8s, CI/CD
  testing/                  Test harnesses
  analytics/                R report scripts
  logo/                     Brand assets
```

16.3 Appendix C: Quick Reference

```
docker compose up -d
docker compose logs -f <service-name>
cd backend/combined && python main.py
cd backend && pytest
cd apps/web && npm run dev
cd apps/mobile && flutter run
cd apps/mobile && flutter build apk
```

Listing 29: Common Development Commands